Fix Guide: JUCE 8 WebView plugin plays a "generic" sound in the DAW instead of its own sound

How to use this document: paste it in full into AI Studio, together with the source of the plugin you want fixed. It describes a bug and its fix that were confirmed in the "Photo Synth / Paint a Synth" plugin. The fix was tested headlessly and then confirmed by the owner in GarageBand/Logic on macOS. Follow the steps **in order**. Do not skip the diagnosis step (Section 2): it shows whether the plugin really has this bug before you change anything.

---

0. Architecture these plugins share

- **C++ side (JUCE 8):** an `AudioProcessor` that holds an `AudioProcessorValueTreeState` (APVTS) and a synth engine (`juce::Synthesiser` + custom voices), plus an `AudioProcessorEditor` that hosts a `juce::WebBrowserComponent`.
- **UI side (React + TypeScript, built with Vite):** loaded into that WebBrowserComponent. The UI does the "unique" work (photo analysis, morph, colour points, presets, effects) and also contains its **own browser synth** (a WebAudio engine, e.g. `WebAudioEngine.ts`) used for the web preview in AI Studio.
- **A bridge file** (e.g. `src/audio/PluginBridge.ts`) that is supposed to send UI changes to C++ through JUCE native functions registered in the editor with `WebBrowserComponent::Options::withNativeFunction('...')`.

1. Symptom and root cause

Symptom

- In the web preview the plugin sounds unique.
- In a DAW (VST3/AU), either the sound is generic/default and changing the photo, presets or knobs has no audible effect, or the unique sound only plays from the on-screen keys and DAW MIDI plays a generic sound.
- If someone "fixes" the double-sound problem by muting the browser WebAudio engine in plugin mode, the unique sound **disappears completely**. This is what happened in Photo Synth, and it is the key clue.

Root cause (confirmed by reproduction)

1. **JUCE 8 does not put `getNativeFunction` on `window`.** JUCE 8 injects only this into the page:

   ```
   window.__JUCE__ = {
     postMessage: <native>,
     initialisationData: { __juce__functions: [...names...], __juce__platform: [...], ... },
     backend: { addEventListener(id, fn), removeEventListener(token), emitEvent(id, obj) }
   }
   ```

   `getNativeFunction()` is defined only in JUCE's **optional frontend JS library** (`modules/juce_gui_extra/native/j` published as npm `juce-framework-frontend`). If the app does not bundle that library, the function does not exist.

2. The bridge looked for `window.__JUCE__.getNativeFunction` / `window.__JUCE__.backend.getNativeFunction` / `window.Juce.getNativeFunction`. It found nothing, so **every call to C++ was silently dropped**. That includes `syncPatch`, `setParameter`, `noteOn`, `noteOff`, `syncSlotImage` and `getInitialState`.

3. On top of that, the full-patch sync only sent when `nativeFunctions.has('syncPatch')` was true, with no fallback.

4. Result: the C++ engine **never received the photo-derived patch** and always played its default patch. The unique sound people heard inside the plugin was the **browser WebAudio engine running inside the WebView**. That audio does not go through the DAW's mixer, cannot be recorded, and does not respond to DAW MIDI.

5. Muting the browser engine removed the only source of the unique sound, and only the C++ default ("generic") sound was left.

The correct fix is to make the bridge actually reach C++, so that the C++ engine plays the unique sound. Do **not** route DAW MIDI to the browser engine.

---

2. Diagnosis — check that the plugin has this bug (do this first)

2a. Static checks

1. Search the UI source for `getNativeFunction`. If the bridge uses it, continue.
2. Check `package.json` and the source for an import of `juce-framework-frontend`, or a vendored copy of JUCE's `index.js` (look for `class PromiseHandler` or `"__juce__invoke"`). **If it is NOT bundled, the plugin has this bug.**
3. Search the editor `.cpp` for `withNativeFunction(`. List every registered name (for Photo Synth: `setParameter, syncPatch, noteOn, noteOff, openFileChooser, syncSlotImage, getInitialState, getWaveformData`).
4. Find the place where the patch is sent (e.g. `syncFullPatch`). Check whether it is guarded by something like `nativeFunctions.has(...)` that will never be true.

2b. Reproduction harness (Node, no DAW needed)

Run the real bridge code against an exact copy of what JUCE injects, and count what actually gets posted to C++:

1. Copy JUCE's injected script out of `modules/juce_gui_extra/misc/juce_WebBrowserComponent.cpp`. It is the raw string `R"( ... )"` that starts with `if (typeof window.__JUCE__ === "undefined")` and ends with `window.__JUCE__.backend = new Backend();`. Save it as `juce_injected.js`.

2. Bundle the bridge for Node:

   ```
   npx esbuild entry.ts --bundle --platform=node --format=cjs --outfile=bridge.cjs --define:import.met
   ```

   where `entry.ts` contains `import { pluginBridge } from '<path>/src/audio/PluginBridge';` `(globalThis as any).pluginBridge = pluginBridge;`

3. `run.cjs`:

   ```
   const fs = require('fs'); const posted = [];
   global.window = global; global.location = { search: '' };
   window.__JUCE__ = { postMessage: (s) => posted.push(JSON.parse(s)) };        // what C++ receives
   window.__JUCE__.initialisationData = { __juce__platform: ['macOS'],
     __juce__functions: ['setParameter','syncPatch','noteOn','noteOff','openFileChooser','syncSlotImag
     __juce__registeredGlobalEventIds: [], __juce__sliders: [], __juce__toggles: [], __juce__comboBoxe
   eval(fs.readFileSync(__dirname + '/juce_injected.js', 'utf8'));
   window.__PHOTO_SYNTH_HYBRID__ = true; window.__PHOTO_SYNTH_NATIVE_HOST__ = true;  // whatever flag
   require('./bridge.cjs');
   (async () => {
     const b = window.pluginBridge; await b.initNativeBridge();
     b.syncFullPatch({ engineType: 'hybrid_wavetable', cutoffOffset: 900, effects: {} },
                     { timbreDna: 0.8, brightness: 0.9, saturation: 0.7, complexity: 0.6 });
     b.setParameter('cutoff', 1234); b.noteOn(60, 100); b.noteOff(60);
   ```

```
  b.syncSlotImage('A', '', 'photo.png');
  await new Promise(r => setTimeout(r, 200));
  console.log('invoke calls:', posted.filter(p => p.eventId === '__juce__invoke').map(p => p.payloa
  process.exit(0);
})();
```

4. **Buggy result (what Photo Synth printed before the fix):**

   ```
   isNative: true
   messages posted to C++: 0 invoke calls: []
   ```

   Zero messages means the plugin has this bug.

---

3. The fix, part 1 — the bridge (`PluginBridge.ts`) speaks the JUCE 8 protocol directly

The protocol (copied from JUCE 8.0.4 `index.js`; must match exactly)

- **JS → C++:** `window.__JUCE__.backend.emitEvent("__juce__invoke", { name: "<functionName>", params: [arg1, arg2, ...], resultId: <unique integer> })`
- **C++ → JS:** the backend emits the event `"__juce__complete"` with payload `{ promiseId: <same integer>, result: <value passed to completion(...)> }`.
- Field names are exact: `name`, `params` (always an **array**), `resultId` going out, and `promiseId`, `result` coming back.

Implementation requirements

1. Add the state: `nextPromiseId = 0` and `pendingCalls: Map<number, {resolve, timer}>`.
2. **Register the completion listener once per backend object:**

   ```
   backend.addEventListener('__juce__complete', (payload) => {
     const pending = this.pendingCalls.get(payload?.promiseId);
     if (pending) { clearTimeout(pending.timer); this.pendingCalls.delete(payload.promiseId); pending.
   });
   ```

3. Write `callNative(name, ...args): Promise<any>` that **never rejects**, in this order:
   1. If a real `getNativeFunction` exists (on `window.__JUCE__`, `.backend` or `window.Juce`), use it, wrapped in a timeout.
   2. Otherwise, if `window.__JUCE__.backend.emitEvent` exists: make sure the completion listener is registered, take `resultId = nextPromiseId++`, store `{resolve, timer}`, then call `backend.emitEvent('__juce__invoke', { name, params: args, resultId })`. If `emitEvent` throws, clean up and resolve `undefined`.
   3. Otherwise use the legacy custom bridge object, if the project has one.
   4. Otherwise `console.warn` that the call was dropped and resolve `undefined`. **Never fail silently.**
4. **Timeouts** (so the UI never hangs if C++ doesn't answer): default 4000 ms; `syncSlotImage` 15000 ms and `getInitialState` 15000 ms (large images); `openFileChooser` 10 minutes (the user is browsing). On timeout: delete the pending entry, `console.warn`, resolve `undefined`.
5. Write `hasNativeTransport()`, which returns true if `getNativeFunction` exists or `backend.emitEvent` is a function. In `initNativeBridge()`, when in plugin mode, poll up to 25 × 40 ms until `hasNativeTransport()` is true (the backend can be injected after the bundle runs). Then register the completion listener and the other backend event listeners (waveform, parameterChanged, bpmChanged) **only once** (guard with a flag).

6. **Every native call must go through `callNative`:** `setParameter`, `syncPatch`, `noteOn`, `noteOff`, `syncSlotImage`, `getInitialState`, `getWaveformData`, `openFileChooser`. Remove every code path that checks `nativeFunctions.has(...)` before sending.
7. **`syncFullPatch(patch, metrics, morph):`** in plugin mode it must **always** `return this.callNative('syncPatch', paramMap)`. In the web preview it updates the browser engine instead.
8. **Build `paramMap` in the units of the C++ APVTS parameters (real units, not 0–1),** and use the exact APVTS parameter IDs. Photo Synth's map:
   - `cutoff` (Hz, default 2400), `resonance` (1.2), `lfoRate` (2.0), `lfoDepth` (0.25), `attack` (s, 0.03), `decay` (0.35), `sustain` (0–1, 0.7), `release` (0.45), `drive` (0.0), `limiter` (1 unless explicitly false), `volume` (dB, −3), `threshold` (−3)
   - `archetype`: **0** = analog_brass, **1** = digital_fm_bells or fm_square_bell, **2** = hybrid_wavetable
   - Photo metrics: `color = clamp01(timbreDna) * 360`, `brightness = clamp01(brightness) * 100`, `saturation = clamp01(saturation) * 100`, `complexity = clamp01(complexity) * 100`
   - `morph` (0–1), when given
   - Effects: `<fx>Enabled` (0/1) plus the mix and time values, for delay, reverb, chorus, phaser, flanger and distortion. For example `delayTime` in ms (clamped 10–1500) and `delayFeedback` clamped to 0–0.95. A mix is 0 when that effect is off.
   - Replace every value that is NaN or not a number with its default before sending.
   - Keys with no matching APVTS parameter are harmless: C++ ignores them.
   - **Adapt the IDs, defaults and scalings to each plugin's own `createParameterLayout()`.** Read that function and map one-to-one.
9. `noteOn` sends `(Math.round(note), velocity in 0..1)`. If a MIDI-style velocity (>1) arrives, divide it by 127.
10. Oscilloscope: poll `getWaveformData` at most every 33 ms, with one request in flight at a time. Until the first buffer arrives, show a flat line (fill with 128).

4. The fix, part 2 — the browser engine must stay silent in plugin mode (no double synth)

In `WebAudioEngine.ts`, add:

```
export function isPluginHostEnvironment(): boolean {
  if (typeof window === 'undefined') return false;
  const w = window as any;
  if (w.__PHOTO_SYNTH_HYBRID__ || w.__PHOTO_SYNTH_NATIVE_HOST__ || w.PhotoSynthNative) return true;   /
  if (w.__JUCE__?.backend || w.__JUCE__?.getNativeFunction || w.Juce?.getNativeFunction) return true;
  try { const q = new URLSearchParams(window.location.search); return q.get('hybrid') === '1' || q.get(
}
```

- Keep this function inside `WebAudioEngine.ts` (importing the bridge there would create a circular import).
- Evaluate it **on every call**, not once at load, because `__JUCE__` may be injected late.
- Return early from `initAudio()` and `noteOn()` when it is true, so no AudioContext is ever created inside the plugin.
- In the bridge, call `globalAudioEngine.*` (noteOn, noteOff, updateParameters, panic) **only when not in plugin mode**.
- **Important ordering:** only mute the browser engine **together with** the bridge fix from Section 3. Muting it alone causes exactly the "only generic sound" regression.

5. The fix, part 3 — the UI (`App.tsx`) must always keep C++ in sync

1. **Startup (mount effect):**

- In the web preview, prime the browser engine with the default patch.
- In plugin mode, run this sequence:
  1. `await pluginBridge.initNativeBridge()`
  2. `init = await pluginBridge.getInitialState()`
  3. If `init.bpm > 0`, apply it.
  4. If `init.slotADataUrl` / `init.slotBDataUrl` are real data URLs (length > 100), re-run the photo analysis on them (`processImageSourceToData`) and use them. Otherwise use the default slots.
  5. If `init.hasState` is true, use `init.morph` and overlay `init.parameters` (real units) onto the interpolated patch: archetype → engineType, cutoff, resonance, LFO, volume, ADSR, limiter, threshold, `<fx>Enabled`.
  6. Set the React state (slots, preview, morph, patch, baseline, metrics).
  7. **Push to C++:** `syncSlotImage` for each slot that C++ did not already hold, then `syncFullPatch(basePatch, baseMetrics, morph)`. Await them all.
  8. Then set `nativeReadyRef.current = true` and mark audio as started.
- The bug variant to remove: initial sync calls wrapped in `if (!isNativeEnvironment())`. That wrapper skips the sync exactly when it is needed.

2. **Safety net:**

```
useEffect(() => {
  if (!pluginBridge.isNativeEnvironment() || !nativeReadyRef.current || !patch || isAutoSweeping) r
  clearTimeout(timer.current);
  timer.current = setTimeout(() => pluginBridge.syncFullPatch(patch, metrics, morphValue), 30);
}, [patch, metrics, morphValue, isAutoSweeping]);
```

   This catches effect toggles, effect params, presets, colour points and resets, even in handlers that forgot to sync.

3. **Explicit syncs:**
   - On slot load or select (`syncSlotImage` + `syncFullPatch`), morph change, the auto-sweep loop (~20 Hz, throttled), parameter changes, presets and full-preset load.
   - Pass the morph value into `syncFullPatch`.
   - Wrap every `globalAudioEngine.updateParameters(...)` in `if (!pluginBridge.isNativeEnvironment())`.

6. The fix, part 4 — C++ side

6a. Native functions return real results (`PluginEditor.cpp`)

Every `withNativeFunction` lambda **must call `completion(...)` exactly once on every path**, otherwise the JS promise only resolves when its timeout runs out. Keep the lambdas thin and put the logic in the processor so it can be unit-tested:

```
.withNativeFunction ("syncPatch", [this](const juce::Array<juce::var>& args, auto completion) {
    const int numSet = args.size() >= 1 ? audioProcessor.applyFrontendPatch (args[0]) : 0;
    completion (juce::var (numSet));
})
.withNativeFunction ("syncSlotImage", [this](const juce::Array<juce::var>& args, auto completion) {
    if (args.size() >= 2) {
        completion (juce::var (audioProcessor.syncSlotImageFromFrontend (args[0].toString(), args[1].to
                                                                         args.size() >= 3 ? args[2].toS
        return;
    }
    completion (juce::var (false));
})
.withNativeFunction ("getInitialState", [this](const juce::Array<juce::var>&, auto completion) {
```

```
    completion (audioProcessor.getInitialStateForFrontend());
})
```

Also keep `.withNativeIntegrationEnabled()` and the `.withUserScript("window.__<PLUGIN>_HYBRID__ = true; ...")` flag on the Options.

6b. `applyFrontendPatch(const juce::var&)` (processor)

- Iterate the object's properties.
- Accept the WebAudio-style alias names: `cutoffOffset→cutoff`, `attackTime→attack`, `decayTime→decay`, `sustainLevel→sustain`, `releaseTime→release`, `masterVolume→volume`, `limiterEnabled→limiter`.
- Skip values that are not numbers or bools, and values that are not finite.
- For each key that matches an APVTS parameter: `param->setValueNotifyingHost(param->convertTo0to1(value))`.
- **Do NOT use a "value 1 means normalised, so multiply" heuristic.** The old code did, and it turned a brightness of 0.8 % into 80 %. The bridge always sends real units.
- If `morph` changed, call `updateMorphState()` and `updateBlendedImage()`.
- If anything was set, set `frontendStateKnown = true`. Return the count.

6c. `syncSlotImageFromFrontend(slot, dataUrl, name)` (processor)

- Decode the image: take the text after the first comma, then `juce::Base64::convertFromBase64` into a `MemoryOutputStream`, then `juce::ImageFileFormat::loadFrom(data, size)`. This auto-detects PNG, JPEG and GIF; the UI usually sends canvas JPEGs.
- If the image is invalid, return false.
- Otherwise store the data URL (`slotAFrontendDataUrl` / `slotBFrontendDataUrl`) and call `loadSlotAImage` / `loadSlotBImage`, which **re-run the C++ image analysis** and publish it. Return true.

6d. `getInitialStateForFrontend()` (processor)

Return an object with these properties: - `bpm`, `isNativeHost: true`, `hasState: frontendStateKnown`, `slotAName`, `slotBName` - `slotADataUrl` / `slotBDataUrl`: **only the photos that came from the UI or a saved project**, never the C++ procedural preset renders - `morph` - `parameters`: every `RangedAudioParameter` in **real units**, i.e. `p->convertFrom0to1(p->getValue())`, not normalised 0–1

6e. Save and reopen the project with the user's photos

- `getStateInformation`: save the APVTS XML plus attributes `slotAName`, `slotBName`, `currentBpm`, and `slotADataUrl` / `slotBDataUrl` when they are not empty.
- `setStateInformation`:
  1. Restore the APVTS, names and BPM.
  2. Clear the stored data URLs, then call `syncSlotImageFromFrontend` with each saved URL. If there is none, or it fails, fall back to the factory preset by name.
  3. Set `frontendStateKnown = true`.
  4. Only publish the analysis snapshots. **Do not touch voices from the message thread.**

6f. Engine robustness (already present in Photo Synth; check that other plugins have it too)

1. **Constructor:**
   1. Create the engine and 16 voices.
   2. Cache the APVTS raw-parameter pointers.
   3. Analyse the default slot images.
   4. `applyAnalysisToParameters(currentAnalysis)`, which **includes writing archetype into the APVTS**.
   5. `pushParametersToEngine()`.

The plugin then plays the photo engine even before the editor opens. Before this change, the archetype was only set by the UI, so DAW MIDI without the editor got archetype 0.

2. **`prepareToPlay`:** `engine.prepare(sampleRate, blockSize)`, which validates that every voice is the custom voice. Then `pushParametersToEngine()`, then reset the UI MIDI FIFO, then prepare the effects and the limiter.
3. **`processBlock`, in this order:**
   1. `buffer.clear()`
   2. Read the host BPM from the play head.
   3. **`pushParametersToEngine()` (APVTS atomics → `VoiceParams` → `engine.setPatch`) before any MIDI is handled**
   4. Drain the on-screen-key notes from a lock-free `juce::AbstractFifo` into the host `MidiBuffer`.
   5. `engine.render(buffer, midi, 0, n)`, so host MIDI and UI notes go to the same voices.
   6. Effects, then master gain.
   7. Scope capture with a try-lock.
4. **Realtime safety:**
   - No `juce::String` or `ImageAnalysisResult` copies on the audio thread. Use a POD `AnalysisSnapshot` published under a `SpinLock`; the audio thread only *try-locks* it.
   - UI `noteOn` / `noteOff` push into the FIFO (writers serialise on a SpinLock the audio thread never takes).
   - BPM is `std::atomic<float>`.
5. `buildVoiceParams()` maps `archetype` 0/1/2 to the engine archetypes. Any other value falls back to the image analysis archetype.

6g. Build settings (`CMakeLists.txt`, inside `juce_add_plugin`)

Use `IS_SYNTH TRUE`, `NEEDS_MIDI_INPUT TRUE`, `AU_MAIN_TYPE kAudioUnitType_MusicDevice`, and add `VST3_CATEGORIES Instrument Synth` so VST3 hosts list the plugin as an instrument and route MIDI to it.

6h. Mirrored source folders

If the repo has a duplicate source folder (Photo Synth had `Source/` as a byte-identical copy of `plugin/Source/`), copy every C++ edit to both.

---

7. Verification (all of these passed for Photo Synth before it was shipped)

1. **Bridge harness (Section 2b) after the fix.** Expected output:

   ```
   messages posted to C++: 6 invoke calls: ["getInitialState","syncPatch","setParameter","noteOn","not
   ```

   Also assert:

   - every payload is `{name, params:[...], resultId}` and the resultIds are unique
   - `syncPatch` has `archetype: 2` for hybrid_wavetable and the scaled metrics (color 288, brightness 90, saturation 70, complexity 60)
   - `noteOn` velocity is normalised to 0–1
   - a simulated reply `window.__JUCE__.backend.emitByBackend('__juce__complete', JSON.stringify({promis 0, result: {...}}))` resolves `getInitialState`
   - a call with no reply resolves `undefined` after the timeout

   Photo Synth result: **19/19 PASS**.

2. **Headless C++ test.** Build against the plugin's shared-code library, create the processor, `prepareToPlay(48000, 512)`. Then:

   - Apply two very different patches through `applyFrontendPatch` (for example archetype 0 with low brightness and cutoff, versus archetype 2 with high brightness and cutoff), render a held note

through `processBlock`, and measure RMS and spectral centroid. **The two must differ clearly.** Photo Synth: centroid went from about 422 Hz to about 10 kHz. 16/16 checks passed.
- Call `syncSlotImageFromFrontend` with two different encoded images and check that the analysis or sound changes.
- Check that host MIDI alone (no UI call at all) produces sound.

3. **Builds:** `npx tsc --noEmit` (or `npm run build`) for the UI; the C++ build for the plugin.
4. **In a real DAW (the owner must do this on their own machine):**
   - Load the AU/VST3 and play from DAW MIDI **and** from the on-screen keys; both must sound the same.
   - Change the photo, morph, presets, colour points and effects: each must audibly change the sound.
   - The oscilloscope moves.
   - Save the project, close it, reopen it: same photos, same sound.
   - On macOS, `auval -v aumu <SubtypeCode> <ManufacturerCode>` passes (Photo Synth: `auval -v aumu Psyn MSTe`).

8. Mistakes to avoid (all happened or nearly happened)

- Muting the browser engine before fixing the bridge. Result: only the generic sound.
- Routing DAW MIDI to the browser WebAudio engine. Its audio bypasses the DAW, so it cannot be recorded or mixed.
- Assuming `getNativeFunction` exists on `window.__JUCE__` in JUCE 8.
- Guarding sends with `nativeFunctions.has(...)`, or wrapping the initial sync in `if (!isNativeEnvironment())`.
- Native-function lambdas that miss `completion(...)` on some path.
- " 1 means normalised" auto-scaling in `syncPatch`. Send real units and map exactly.
  - Photo Synth's `setParameter` lambda still has this heuristic for color/brightness/saturation/complexity. It is harmless there because the UI sends real units, but don't copy it into new plugins.
- Touching voices or the synth from the message thread, or copying `juce::String` objects on the audio thread.
- Declaring it "fixed" from a build passing. Run the harness and the headless audio-difference test.

Alternative to Section 3 (equally valid)

Bundle JUCE's own frontend library instead of hand-writing the protocol: `npm i juce-framework-frontend` (or vendor `modules/juce_gui_extra/native/javascript/index.js`), then `import * as Juce from 'juce-framework-frontend'` and use `Juce.getNativeFunction(name)`. The library must be imported **only when running inside the plugin**, because at import time it expects `window.__JUCE__` to exist. Section 3 avoids that problem, which is why Photo Synth used it. The rest of this guide (Sections 4–7) still applies either way.

Optional: CI artifacts

GitHub's Actions summary may list only one artifact even when every platform built. Check the run's artifacts through the API (`/repos/<owner>/<repo>/actions/runs/<id>/artifacts`). For easy downloads, either add a job that combines the platform zips and writes a `$GITHUB_STEP_SUMMARY` table, or attach the zips to a GitHub (pre-)release.